

Take-Home Task — Shopify Integration

Overview

Build a small Shopify app that a merchant can install, manage through an admin panel, and control through a secondary AI agent. You own both the frontend (UI/UX) and the backend, including the OAuth flow.

You may use any language, framework, and libraries you're comfortable with. Use a Shopify **development store** (free via the Shopify Partner program) to build and test against.

Target Shopify API version: `2026-04` (pin it explicitly in code).

What to Build

1. OAuth Flow

- Let a merchant install the app and authorize it against their store.
- Complete the full OAuth handshake and securely store the resulting access token per shop.
- Handle the `app/uninstalled` webhook (clean up the shop's data/token).

2. Admin Panel (UI/UX + Backend)

A simple web panel the merchant sees after installing. It should include:

- **Connection status** — show the connected store and that auth is working.
- **Actions screen** — trigger the actions from section 3 through forms/buttons, with a confirmation step before any write and a clear success/error result.
- **Activity log** — a list of actions taken (what, when, result).

Design and usability matter. Keep it clean, responsive, and consistent.

3. Actions

Implement at least **three** store actions against the Shopify Admin API, including at least one read and one write. For example:

- List / search products
- Create or update a product
- Adjust inventory for a product
- List recent orders / fulfill an order

Each action must validate input and handle Shopify errors and rate limits gracefully.

4. Secondary AI Agent (with MCP)

Add an AI agent the merchant can chat with to run the actions from section 3.

- The agent must call the actions through **MCP tools** (not by hardcoding API calls in the prompt).
- Read actions may run automatically; **any write action must require explicit human confirmation** before it executes.
- Log the agent's tool calls and results.

You may wrap your own actions as an MCP server, or use an existing Shopify MCP server — your choice, but explain what you picked.

Deliverables

1. A Git repository with your code and clear commit history.
 2. A `README.md` explaining:
 - How to set up and run it locally (env vars, store setup, steps).
 - Which decisions and trade-offs you made and why.
 - What you'd improve or add with more time.
 3. A short screen recording **or** live demo showing: install → OAuth → run an action from the panel → run an action via the AI agent (including the confirmation step for a write).
-

Requirements & Constraints

- Verify request authenticity where Shopify requires it (OAuth callback and webhooks).
 - Never expose or log the access token on the client side.
 - Request only the scopes your actions actually need.
 - Destructive/write actions must be confirmed by a human before running — from both the panel and the agent.
 - Handle Shopify API errors and rate limits without crashing.
-

Evaluation Criteria

Area	What we look for
Correctness	OAuth works end-to-end; actions do what they claim; agent executes real actions via MCP.

Area	What we look for
Security	Token handling, request verification, scope hygiene, human confirmation on writes.
Code quality	Structure, readability, error handling, sensible tests.
UI/UX	Clear, usable, responsive admin panel; good confirmation and error states.
AI/MCP integration	Agent correctly uses tools; write-confirmation gate works; actions logged.
Communication	README clarity, reasoning about trade-offs, demo.

Time & Submission

- Suggested effort: **6-10 hours**. We do not expect production completeness — focus on a working, well-reasoned slice.
- If you run short on time, ship less but make it solid, and note in the README what you'd do next.
- Submit the repo link (and recording, if not doing a live demo) by the agreed deadline.

You're welcome to ask clarifying questions before starting.